

365-Day Java Mastery Roadmap

For a developer with 2 years experience aiming to become top 1% in 4-5 years
10-15 min/day, one small topic at a time — read, code a tiny example, move on.

How to use this:

- Every day, pick ONE topic. Read/watch for 5-7 min, then write 10-15 lines of code testing it. Don't just read.
- Keep a "Today I Learned" (TIL) log — one line per day. After a year you'll have a personal encyclopedia.
- Revisit Month 1-3 topics briefly every 90 days (spaced repetition) — this is what makes it stick for years, not weeks.
- Skip days you already know cold — replace with a "build" day (mini project applying last week's topics).

MONTH 1 — Core Java Re-Foundation (you'll skim fast, but go deep on internals)

1. How the JVM class loading process works (Bootstrap/Extension/App classloader)
2. `==` vs `.equals()` — what really happens in memory
3. String pool, `String.intern()`, why Strings are immutable
4. `StringBuilder` vs `StringBuffer` — internal char array growth
5. Primitive vs wrapper types, autoboxing pitfalls (Integer cache -128 to 127)
6. `final`, `finally`, `finalize()` — and why `finalize` is deprecated
7. Access modifiers deep dive (package-private nuances)
8. Static vs instance — memory allocation differences
9. Static initialization blocks vs instance initialization blocks, execution order
10. Constructors: overloading, chaining (`this()`, `super()`)
11. `instanceof` and pattern matching for `instanceof` (Java 16+)
12. Object class methods: `hashCode()`, `equals()`, `toString()`, `clone()`
13. Deep copy vs shallow copy — implementing `Cloneable` properly
14. Arrays in Java — how they're objects, covariance issues
15. Wrapper class caching and autoboxing performance traps
16. `var` (local type inference) — where it helps, where it hurts readability
17. Enums — full power (fields, methods, abstract methods per constant)
18. Enum singleton pattern — why it's the safest singleton
19. Interfaces: default methods, static methods, private methods (Java 8/9)
20. Functional interfaces — `@FunctionalInterface`, writing your own
21. Nested classes: static nested, inner, local, anonymous — when to use which
22. Records (Java 14+) — replacing boilerplate POJOs
23. Sealed classes (Java 17) — restricting inheritance hierarchies
24. Text blocks (Java 15) — multi-line strings done right
25. Pattern matching for `switch` (Java 21)
26. `Optional<T>` — correct usage, anti-patterns to avoid
27. Checked vs unchecked exceptions — the real design reasoning
28. Custom exceptions — designing an exception hierarchy for an app
29. Try-with-resources and `AutoCloseable`

30. Build day: refactor an old project's model classes using records + sealed types

MONTH 2 — Collections Framework Mastery

31. Collection hierarchy overview (Collection, List, Set, Map, Queue)
32. ArrayList internals — resizing algorithm, amortized cost
33. LinkedList internals — doubly linked list, when it's actually worth using
34. Vector & Stack — legacy, why avoid them
35. HashMap internals — buckets, hashing, treeification (Java 8+)
36. HashMap resize/rehashing mechanics, load factor
37. LinkedHashMap — access order vs insertion order, LRU cache use case
38. TreeMap — Red-Black tree, Comparable vs Comparator
39. HashSet vs LinkedHashSet vs TreeSet
40. ConcurrentHashMap internals — segment locking (old) vs CAS+bins (new)
41. CopyOnWriteArrayList — when it makes sense
42. Queue/Deque: ArrayDeque vs LinkedList as a queue
43. PriorityQueue — heap-based, custom comparator use
44. Iterator vs ListIterator, fail-fast vs fail-safe iterators
45. ConcurrentModificationException — causes and fixes
46. Comparable vs Comparator — multi-field sorting with thenComparing
47. Collections utility class — synchronizedX, unmodifiableX, emptyX
48. Arrays utility class — sort, binarySearch, parallelSort
49. Big-O of every collection operation (make yourself a cheat sheet)
50. Choosing the right collection for a given problem (decision framework)
51. Immutable collections — List.of(), Map.of(), Guava ImmutableList
52. Weak references & WeakHashMap — cache use cases
53. IdentityHashMap — rare but interesting use case
54. EnumMap / EnumSet — performance benefits over regular maps/sets
55. Custom equals/hashCode contract violations and bugs they cause
56. Building a custom LRU cache using LinkedHashMap
57. Building a simple custom HashMap from scratch (educational)
58. Streams recap: intermediate vs terminal operations
59. Collectors: groupingBy, partitioningBy, toMap edge cases
60. Build day: implement an LFU cache + benchmark 3 collection choices

MONTH 3 — Generics, Strings, Exceptions Deep Dive

61. Generics basics — type erasure explained properly
62. Bounded type parameters (<T extends Comparable<T>>)
63. Wildcards: ? extends, ? super, PECS principle
64. Generic methods vs generic classes
65. Why you can't create generic arrays
66. Bridge methods and generics + inheritance gotchas

67. String immutability — security and thread-safety benefits
68. String formatting: `String.format`, `printf`, text blocks with `.formatted()`
69. Regex in Java — Pattern/Matcher, common recipes
70. StringBuilder chaining performance vs string concatenation in loops
71. Character encoding basics — UTF-8 vs UTF-16, common bugs
72. Custom checked exception design — when checked is the right call
73. Exception chaining (`initCause`, wrapping exceptions)
74. Multi-catch blocks, exception hierarchies
75. Try-with-resources with multiple resources, suppressed exceptions
76. Stack traces — reading them fast, `getStackTrace()` programmatically
77. Assertions (`assert` keyword) — when/why they're used
78. Objects utility class — `requireNonNull`, `equals`, `hash`
79. Builder pattern for complex object construction
80. Fluent APIs — designing method chains
81. Defensive copying — protecting mutable state in constructors/getters
82. Varargs — internals and pitfalls (array creation cost, ambiguity)
83. Method overloading resolution rules (widening vs boxing vs varargs priority)
84. `this` escaping during construction — a subtle bug class
85. Immutable class design checklist (final fields, no setters, defensive copies)
86. Value-based classes concept (`Integer`, `LocalDate`, etc.)
87. Java Time API: `LocalDate`/`LocalTime`/`LocalDateTime`/`ZonedDateTime`
88. Duration vs Period, date math done right
89. Legacy Date/Calendar problems (why `java.time` replaced them)
90. Build day: write a small library with a fluent builder + custom exceptions

MONTH 4 — Multithreading & Concurrency (this separates seniors from juniors)

91. Process vs Thread, Java memory model overview
92. Creating threads: `Thread` vs `Runnable` vs `Callable`
93. Thread lifecycle states (`NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, `TERMINATED`)
94. `synchronized` keyword — method vs block level
95. Intrinsic locks (monitor) — how `synchronized` really works
96. `wait()`, `notify()`, `notifyAll()` — classic producer-consumer
97. Deadlock — causes, detection, and a live demo you write yourself
98. Livelock and starvation — how they differ from deadlock
99. `volatile` keyword — visibility guarantee, NOT atomicity
100. Java Memory Model (JMM) — happens-before relationship
101. Atomic classes: `AtomicInteger`, `AtomicReference`, CAS operations
102. `ReentrantLock` vs `synchronized` — `tryLock`, fairness, interruptibility
103. `ReadWriteLock` — when read-heavy workloads benefit
104. `CountDownLatch` — use case: wait for N tasks to finish
105. `CyclicBarrier` — use case: N threads sync at a point repeatedly

- 106. Semaphore — limiting concurrent access to a resource
- 107. Exchanger — rare but good to know
- 108. ExecutorService — fixed, cached, single-thread, scheduled pools
- 109. Future and Callable — getting results back from threads
- 110. CompletableFuture basics — supplyAsync, thenApply, thenCompose
- 111. CompletableFuture combinators — allOf, anyOf, exception handling
- 112. ForkJoinPool — work-stealing algorithm, RecursiveTask
- 113. Parallel streams — when they help, when they backfire
- 114. ThreadLocal — use cases and memory leak risks
- 115. ThreadLocal in web servers — request-scoped data pattern
- 116. Producer-consumer pattern with BlockingQueue
- 117. BlockingQueue implementations: ArrayBlockingQueue, LinkedBlockingQueue, SynchronousQueue
- 118. Thread pool sizing — CPU-bound vs I/O-bound formulas
- 119. Common concurrency bugs: race conditions, visibility bugs, unsafe publication
- 120. Virtual Threads (Java 21 Project Loom) — what changes and why it matters
- 121. Structured concurrency (preview) — the future of Java concurrency
- 122. Designing a thread-safe singleton (double-checked locking, holder idiom)
- 123. Immutability as a concurrency strategy (recap + why it's the best default)
- 124. synchronized collections vs java.util.concurrent collections
- 125. Debugging deadlocks with jstack / thread dumps
- 126. Build day: build a multi-threaded web crawler using ExecutorService
- 127. Build day: rebuild it using CompletableFuture
- 128. Build day: rebuild it using Virtual Threads, compare code + performance
- 129. Concurrency interview questions — write your own answers to top 15
- 130. Review + spaced repetition of Month 4 (the hardest month — repeat it)

MONTH 5 — JVM Internals, Memory & Performance

- 131. JVM architecture overview: ClassLoader, Runtime Data Areas, Execution Engine
- 132. Heap structure: Young Gen (Eden, S0, S1), Old Gen
- 133. Stack vs Heap — what lives where, per-thread stacks
- 134. Metaspace (replaced PermGen) — class metadata storage
- 135. Garbage Collection basics — mark, sweep, compact
- 136. GC algorithms: Serial, Parallel, CMS (legacy), G1
- 137. ZGC and Shenandoah — low-pause modern collectors
- 138. Choosing a GC for your workload, common JVM flags (-Xms, -Xmx, -XX:+UseG1GC)
- 139. Object lifecycle & generational hypothesis (why most objects die young)
- 140. Minor GC vs Major GC vs Full GC — cost differences
- 141. Memory leaks in Java — common causes despite having a GC
- 142. WeakReference, SoftReference, PhantomReference explained
- 143. Heap dump analysis with Eclipse MAT or VisualVM
- 144. Reading a GC log — what to look for
- 145. JIT compiler — interpreter vs C1 vs C2, tiered compilation
- 146. Escape analysis and stack allocation optimization

- 147. Inlining, dead code elimination — JIT optimizations that matter
- 148. `jstat`, `jstack`, `jmap`, `jcmd` — the JVM toolbox
- 149. Profiling with `async-profiler` / JFR (Java Flight Recorder)
- 150. CPU profiling vs memory profiling — different tools, different goals
- 151. Class file structure basics (bytecode overview, `javap -c`)
- 152. Bytecode verification and class loading phases (loading, linking, initialization)
- 153. Custom class loaders — real-world use cases (plugins, hot reload)
- 154. `OutOfMemoryError` types — heap, metaspace, stack overflow — root causes
- 155. Tuning thread pool + GC together for a high-throughput service
- 156. JVM startup performance — AppCDS, tiered compilation flags
- 157. Native memory tracking (`-XX:NativeMemoryTracking`)
- 158. Benchmarking correctly with JMH (Java Microbenchmark Harness)
- 159. Common JMH pitfalls (dead code elimination fooling your benchmark)
- 160. Build day: JFR-profile one of your real services, find one optimization

MONTH 6 — Modern Java (8 through 21) Feature Mastery

- 161. Lambda expressions — syntax, effectively final variables
- 162. Method references — 4 types (static, instance, particular object, constructor)
- 163. Stream API fundamentals — creation, laziness
- 164. Stream operations: `map`, `filter`, `reduce`, `flatMap`
- 165. Stream short-circuiting: `anyMatch`, `findFirst`, `limit`
- 166. Collectors deep dive: `joining`, `counting`, `averagingInt`, custom Collector
- 167. Parallel streams internals — `Splitterator`, common ForkJoinPool
- 168. Optional chaining patterns — `map/flatMap/orElseGet` vs `orElse`
- 169. Default & static interface methods — diamond problem resolution rules
- 170. `java.util.function` package — `Supplier`, `Consumer`, `Function`, `Predicate`, `BiFunction`
- 171. Date/Time API recap with formatting (`DateTimeFormatter`)
- 172. Java 9: modules (JPMS) — `module-info.java`, why modularity matters
- 173. Java 9: JShell — REPL for quick experiments
- 174. Java 10: local variable type inference (`var`) deep dive
- 175. Java 11: new String methods (`isBlank`, `strip`, `repeat`, `lines`)
- 176. Java 11: `HttpClient` — modern async HTTP without external libraries
- 177. Java 12-13: switch expressions (preview) groundwork
- 178. Java 14: switch expressions finalized, helpful `NullPointerExceptions`
- 179. Java 15: text blocks finalized
- 180. Java 16: records finalized, pattern matching for `instanceof`
- 181. Java 17 (LTS): sealed classes, deprecate/remove security manager
- 182. Java 18: UTF-8 by default, simple web server for testing
- 183. Java 19-20: virtual threads preview, structured concurrency preview
- 184. Java 21 (LTS): virtual threads finalized, pattern matching for `switch`, record patterns
- 185. Java 21: sequenced collections (`SequencedCollection`, `getFirst`, `getLast`)
- 186. Deconstructing records with pattern matching — real examples
- 187. Comparing Java's evolution to Kotlin/Scala features (what Java borrowed)

- 188. Deprecated & removed features you should stop using (Applets, SecurityManager, etc.)
- 189. Choosing which modern features to use in production code (team readability)
- 190. Build day: rewrite an old Java 8 codebase snippet using Java 21 idioms

MONTH 7 — Spring Core & Spring Boot

- 191. IoC container concept — why dependency injection matters
- 192. Bean lifecycle — instantiation, population, initialization, destruction
- 193. `@Component`, `@Service`, `@Repository`, `@Controller` — semantic differences
- 194. Constructor injection vs field injection vs setter injection (and why constructor wins)
- 195. `@Autowired` resolution — by type, by name, `@Qualifier`, `@Primary`
- 196. Bean scopes: singleton, prototype, request, session
- 197. `@Configuration` and `@Bean` — Java-based config
- 198. Application context vs bean factory
- 199. `@PostConstruct` / `@PreDestroy` lifecycle hooks
- 200. Circular dependency problems and how Spring resolves (or fails) them
- 201. AOP basics — proxies, `@Aspect`, pointcuts, advice types
- 202. JDK dynamic proxies vs CGLIB proxies — when Spring uses which
- 203. Spring Boot autoconfiguration — how `@EnableAutoConfiguration` works
- 204. `application.properties` vs `application.yml` — profiles (`@Profile`)
- 205. Spring Boot starters — what's actually inside `spring-boot-starter-web`
- 206. Embedded Tomcat/Jetty/Undertow — how Boot runs without a WAR
- 207. `@RestController`, `@RequestMapping`, `@GetMapping` etc. — request handling
- 208. Request/response binding — `@RequestBody`, `@PathVariable`, `@RequestParam`
- 209. Validation — `@Valid`, Bean Validation (JSR 380) annotations
- 210. Exception handling — `@ControllerAdvice`, `@ExceptionHandler`
- 211. Custom filters and interceptors — request lifecycle hooks
- 212. `HandlerInterceptor` vs `Filter` — where each fits
- 213. Spring Boot Actuator — health checks, metrics, custom endpoints
- 214. Externalized configuration — `@ConfigurationProperties`, environment abstraction
- 215. Spring profiles for dev/test/prod environment separation
- 216. Spring Events — `ApplicationEvent`, `@EventListener`, async events
- 217. Spring's `@Async` — thread pool config, pitfalls (self-invocation)
- 218. `@Scheduled` tasks — cron expressions, fixed rate/delay
- 219. Spring Boot testing — `@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest`
- 220. Build day: build a small REST API from scratch with validation + exception handling

MONTH 8 — Persistence: JPA/Hibernate, SQL & Databases

- 221. JDBC fundamentals — `Connection`, `Statement`, `PreparedStatement`, `ResultSet`
- 222. SQL injection — how `PreparedStatement` prevents it
- 223. Connection pooling — HikariCP internals, pool sizing formula
- 224. JPA vs Hibernate — spec vs implementation

- 225. Entity mapping — @Entity, @Table, @Id, @GeneratedValue strategies
- 226. Relationships: @OneToMany, @ManyToOne, @ManyToMany, @OneToOne
- 227. Fetch types: LAZY vs EAGER — N+1 query problem explained
- 228. Solving N+1 with JOIN FETCH, @EntityGraph, batch fetching
- 229. Cascade types — PERSIST, MERGE, REMOVE and their real effects
- 230. First-level cache (persistence context) vs second-level cache
- 231. @Transactional — propagation types (REQUIRED, REQUIRES_NEW, NESTED)
- 232. Isolation levels — READ_UNCOMMITTED to SERIALIZABLE, what each prevents
- 233. Optimistic locking (@Version) vs pessimistic locking
- 234. Dirty checking — how Hibernate detects changes without explicit save()
- 235. Entity lifecycle states: transient, persistent, detached, removed
- 236. Spring Data JPA — derived query methods, @Query, native queries
- 237. Pagination & sorting with Pageable
- 238. Database indexing — B-tree basics, when an index helps vs hurts
- 239. Composite indexes — column order matters, covering indexes
- 240. EXPLAIN ANALYZE — reading a query plan (Postgres/MySQL)
- 241. Normalization (1NF-3NF) vs denormalization tradeoffs
- 242. Database transactions — ACID properties explained with examples
- 243. Deadlocks at the database level — causes and prevention
- 244. Database migrations — Flyway/Liquibase, why never edit schema manually
- 245. Connection leaks — symptoms, detection, prevention
- 246. SQL joins deep dive: INNER, LEFT, RIGHT, FULL, self-join, cross join
- 247. Window functions — ROW_NUMBER, RANK, PARTITION BY
- 248. CTEs (Common Table Expressions) — recursive queries
- 249. Query optimization checklist (indexes, avoid SELECT *, batch operations)
- 250. NoSQL basics — when Mongo/Redis/Cassandra beat relational DBs
- 251. Redis fundamentals — caching patterns (cache-aside, write-through)
- 252. Cache invalidation strategies — the "two hard things in CS" joke, seriously
- 253. Database sharding & partitioning concepts
- 254. Read replicas & replication lag — consistency tradeoffs
- 255. Build day: profile a slow query in a real project, fix with indexing
- 256. Build day: implement Redis caching for a hot endpoint

MONTH 9 — Design Patterns, SOLID & Clean Code

- 257. SOLID: Single Responsibility Principle — with a before/after refactor
- 258. SOLID: Open/Closed Principle — extension without modification
- 259. SOLID: Liskov Substitution Principle — subtle violations to watch for
- 260. SOLID: Interface Segregation Principle — fat interfaces problem
- 261. SOLID: Dependency Inversion Principle — the "D" in DI frameworks
- 262. Creational: Singleton (all safe implementations)
- 263. Creational: Factory Method vs Abstract Factory
- 264. Creational: Builder pattern (recap + Lombok @Builder)
- 265. Creational: Prototype pattern

- 266. Structural: Adapter pattern — real use case (legacy API wrapping)
- 267. Structural: Decorator pattern — vs inheritance
- 268. Structural: Facade pattern — simplifying complex subsystems
- 269. Structural: Proxy pattern — how Spring AOP literally uses this
- 270. Structural: Composite pattern — tree structures
- 271. Structural: Bridge pattern
- 272. Behavioral: Strategy pattern — swap out `if/else` chains
- 273. Behavioral: Observer pattern — how Spring events use this
- 274. Behavioral: Command pattern — undo/redo, task queues
- 275. Behavioral: Chain of Responsibility — how servlet filters use this
- 276. Behavioral: Template Method — how `JdbcTemplate` uses this
- 277. Behavioral: State pattern — modeling workflows/state machines
- 278. Behavioral: Visitor pattern — rare but shows up in compilers/ASTs
- 279. Behavioral: Mediator pattern
- 280. Anti-patterns: God Object, Spaghetti Code, Anemic Domain Model
- 281. DRY, KISS, YAGNI — when to apply and when they conflict
- 282. Law of Demeter — "don't talk to strangers" in code
- 283. Code smells checklist — long methods, feature envy, shotgun surgery
- 284. Refactoring techniques — extract method, extract class, replace conditional with polymorphism
- 285. Clean code naming — the cost of a bad variable name
- 286. Writing self-documenting code vs over-commenting
- 287. Domain-Driven Design basics — entities, value objects, aggregates
- 288. Repository pattern — abstracting persistence properly
- 289. CQRS pattern basics — separating reads from writes
- 290. Build day: refactor a messy class in a real project applying 3 patterns from this month

MONTH 10 — Data Structures & Algorithms (for interviews + real engineering)

- 291. Big-O notation refresher — time/space complexity analysis
- 292. Arrays & Strings — two-pointer technique
- 293. Sliding window technique
- 294. Linked lists — reverse, detect cycle (Floyd's algorithm)
- 295. Stacks — valid parentheses, next greater element
- 296. Queues — implement using two stacks
- 297. Binary search — and its many variants (rotated array, first/last occurrence)
- 298. Recursion fundamentals — base case design, call stack visualization
- 299. Backtracking — permutations, combinations, N-Queens
- 300. Trees — binary tree traversals (in/pre/post-order, iterative + recursive)
- 301. Binary Search Trees — insert, delete, validate
- 302. Tree — level order traversal (BFS), height, diameter
- 303. Balanced trees concept — AVL, Red-Black (know the idea, not the full impl)
- 304. Heaps — build a min-heap/max-heap from scratch

- 305. Heap use cases — top-K problems, median finder
- 306. Tries — implement one, use case: autocomplete
- 307. Graphs — representations (adjacency list vs matrix)
- 308. Graph traversal — BFS and DFS with real problems
- 309. Topological sort — course scheduling problem
- 310. Union-Find (Disjoint Set) — cycle detection, connected components
- 311. Dijkstra's algorithm — shortest path
- 312. Dynamic Programming — 1D DP (climbing stairs, house robber)
- 313. Dynamic Programming — 2D DP (grid paths, edit distance)
- 314. Knapsack problem (0/1 and unbounded)
- 315. Greedy algorithms — interval scheduling, when greedy works
- 316. Sorting algorithms — quicksort, mergesort implementation from scratch
- 317. Sorting — when to use which (stability, in-place, time complexity)
- 318. Hashing — designing a good hash function, collision handling
- 319. Bit manipulation — common tricks (XOR swap, count set bits)
- 320. Solve 2-3 medium LeetCode problems weekly from here on (ongoing habit)
- 321-350. 30 mixed practice days: 1 DSA problem/day covering arrays, trees, graphs, DP (rotate through topics above)

MONTH 11 — System Design & Microservices

- 351. What is system design — functional vs non-functional requirements
- 352. Scalability basics — vertical vs horizontal scaling
- 353. Load balancing — round robin, least connections, consistent hashing
- 354. Caching strategies at scale — CDN, application cache, DB cache layers
- 355. Database scaling — replication, sharding revisited at scale
- 356. CAP theorem — consistency, availability, partition tolerance tradeoffs
- 357. Microservices vs monolith — real tradeoffs, not hype
- 358. Service decomposition — bounded contexts (tie back to DDD)
- 359. API Gateway pattern — routing, auth, rate limiting at the edge
- 360. Service discovery — client-side vs server-side (Eureka, Consul)
- 361. Inter-service communication — REST vs gRPC vs messaging
- 362. Message queues — Kafka fundamentals (topics, partitions, consumer groups)
- 363. Message queues — RabbitMQ fundamentals (exchanges, queues, routing)
- 364. Event-driven architecture — event sourcing basics
- 365. Capstone: design a real system end-to-end (e.g. URL shortener, rate limiter, or notification service) — write the full design doc

MONTH 12 (BONUS — if you want 13 months) — Testing, DevOps, Security, Career

- Unit testing with JUnit 5 — lifecycle annotations, assertions, parameterized tests
- Mockito — mocking, stubbing, verifying interactions

- Integration testing — Testcontainers for real DB/Kafka in tests
- Test pyramid — unit vs integration vs E2E balance
- CI/CD basics — what a pipeline actually does (build, test, deploy stages)
- Docker fundamentals — images, containers, Dockerfile for a Spring Boot app
- Docker Compose — running app + DB + Redis together locally
- Kubernetes basics — pods, deployments, services (conceptual, not deep ops)
- Git mastery — rebase vs merge, interactive rebase, bisect
- OWASP Top 10 — SQL injection, XSS, CSRF explained with fixes
- JWT & OAuth2 — how token-based auth actually works
- Spring Security basics — filter chain, authentication vs authorization
- Logging best practices — structured logging, correlation IDs, log levels
- Observability — metrics, logs, traces (the three pillars)
- Distributed tracing — how a trace ID follows a request across services
- Writing a design doc / RFC — the skill that gets you promoted
- Mock system design interviews — do 4 this month
- Contributing to an open-source Java project — pick one, submit a PR
- Building your technical brand — write 2 blog posts on what you learned this year
- Year-end review: rebuild your Month-1 self-assessment, measure the gap closed

EXTRA MODULE A — System Design Patterns Catalog (every SDE must recognize these)

1. Client-Server pattern — the baseline everything builds on
2. Layered (N-Tier) Architecture — presentation/business/data separation
3. Load Balancer patterns — Layer 4 vs Layer 7, health checks
4. Reverse Proxy pattern — Nginx/HAProxy role in front of services
5. API Gateway pattern — single entry point, cross-cutting concerns
6. Sidecar pattern — service mesh helpers (Envoy, Istio)
7. Circuit Breaker pattern — stop cascading failures (Resilience4j/Hystrix)
8. Bulkhead pattern — isolating failures to one resource pool
9. Retry pattern with exponential backoff + jitter
10. Timeout pattern — every network call needs one, no exceptions
11. Rate Limiting patterns — token bucket, leaky bucket, sliding window
12. Idempotency pattern — idempotency keys for safe retries (payments, orders)
13. Saga pattern — distributed transactions without 2PC (choreography vs orchestration)
14. Two-Phase Commit (2PC) — why it doesn't scale, when it's still used
15. Outbox pattern — reliable event publishing with DB transactions
16. CQRS — command/query responsibility segregation at scale
17. Event Sourcing — storing state as a sequence of events
18. Strangler Fig pattern — migrating a monolith to microservices safely
19. Anti-Corruption Layer — isolating legacy systems from new domain models
20. Backend for Frontend (BFF) — tailored APIs per client type
21. Write-Ahead Log pattern — how databases guarantee durability

22. Leader Election — Raft/Paxos basics, why distributed systems need consensus
23. Consistent Hashing — even data distribution, minimal reshuffling on scale-out
24. Bloom Filters — probabilistic membership checks at scale
25. Rate of change vs snapshot — CDC (Change Data Capture) pattern
26. Fan-out/Fan-in pattern — parallel task dispatch and aggregation
27. Publish-Subscribe pattern vs point-to-point messaging
28. Dead Letter Queue pattern — handling poison messages
29. Materialized View pattern — precomputed views for read-heavy systems
30. Throttling vs Load Shedding — protecting a system under overload
31. Health Check & Readiness/Liveness probes — Kubernetes-native design
32. Blue-Green Deployment vs Canary Release vs Rolling Deployment
33. Feature Flags / Toggle pattern — decoupling deploy from release
34. Multi-region / Active-Active vs Active-Passive architecture
35. Real interview practice: whiteboard 5 of these patterns from memory, no notes

EXTRA MODULE B — Writing Scalable, Production-Grade Code

1. Stateless service design — why statelessness enables horizontal scaling
2. Twelve-Factor App principles — config, dependencies, backing services, logs
3. Designing idempotent APIs (PUT vs POST semantics, idempotency keys)
4. Pagination at scale — offset vs cursor-based pagination
5. Backward-compatible API evolution — additive changes, versioning strategies
6. Contract testing between services (Pact) — avoiding breaking consumers
7. Graceful degradation — fallback responses when a dependency is down
8. Connection pool tuning under load (DB + HTTP client pools)
9. Batching & bulk operations to reduce round-trips
10. Async processing — moving slow work off the request thread (queues, events)
11. Backpressure — what happens when producers outpace consumers
12. Designing for horizontal scale from day one (no local file/session state)
13. Database connection limits vs service replica count — the multiplication trap
14. Hot partition / hot key problems in sharded systems
15. Write amplification — why naive caching/logging can hurt at scale
16. Observability-first coding — structured logs, metrics, and trace IDs by default
17. SLA vs SLO vs SLI — defining and measuring reliability properly
18. Error budgets — balancing velocity and reliability
19. Capacity planning basics — back-of-envelope estimation (QPS, storage, bandwidth)
20. Cost-aware engineering — how architecture choices affect cloud spend
21. Defensive programming for external calls — null checks, timeouts, retries, circuit breakers together
22. Config-driven behavior over hardcoded constants (feature flags, dynamic config)
23. Zero-downtime deployments — DB migration compatibility (expand/contract pattern)
24. Testing at scale — contract tests, chaos testing basics, load testing (k6/Gatling)
25. Code review habits that scale a team, not just a codebase
26. Writing runbooks and playbooks for on-call incidents
27. Postmortems — blameless root cause analysis practice

28. Technical debt tracking — when to pay it down vs when to accept it
29. Designing for observability during an outage, not just steady state
30. Build day: take one existing endpoint and make it idempotent + rate-limited + circuit-broken

EXTRA MODULE C — Extra Things Every SDE Should Know (grab-bag, high value)

1. HTTP deep dive — status codes that actually matter, headers, caching (ETag, Cache-Control)
2. REST vs GraphQL vs gRPC — real tradeoffs, not hype
3. TCP vs UDP — when each is chosen, TCP handshake basics
4. DNS resolution — how a URL becomes an IP, TTLs and propagation
5. TLS/SSL basics — handshake overview, why HTTPS matters for design
6. Networking basics for backend devs — ports, sockets, keep-alive
7. CAP theorem revisited with real DB examples (Cassandra vs traditional RDBMS)
8. Consistency models — strong, eventual, causal consistency
9. Idempotent vs non-idempotent HTTP methods (GET/PUT/DELETE vs POST)
10. Web security basics — CORS, CSRF, XSS, same-origin policy
11. Authentication vs Authorization — the difference, and where each lives in an architecture
12. API design best practices — resource naming, versioning, error response shape
13. Message serialization formats — JSON vs Protobuf vs Avro, tradeoffs
14. Idempotency in messaging systems — exactly-once vs at-least-once delivery
15. Kafka deeper — consumer offsets, rebalancing, exactly-once semantics
16. Monitoring stack basics — Prometheus + Grafana conceptually
17. Logging pipeline basics — ELK/EFK stack conceptually
18. Cloud fundamentals — compute, storage, networking primitives (any major provider)
19. Infrastructure as Code — Terraform concept (declarative infra)
20. Cost of a network call vs a local call — why this changes design decisions
21. Estimating system requirements in an interview (users, QPS, storage math)
22. Writing a one-page technical design doc that gets approved fast
23. Negotiating tradeoffs — consistency vs availability, latency vs cost, in plain language
24. How to read someone else's large unfamiliar codebase quickly
25. Debugging production issues systematically (reproduce, isolate, fix, verify, document)
26. Interviewing skill — explaining your own project's architecture crisply in 2 minutes
27. Staying current — following JEPs (Java Enhancement Proposals), release notes, key blogs
28. Building a portfolio project that demonstrates system design, not just CRUD
29. Understanding your own service's dependency graph end-to-end
30. Career habit: write one design doc and one postmortem-style retro every quarter, for real work

How to sustain this for 4-5 years (after year 1)

- **Year 2:** Go deep on ONE specialization — distributed systems, performance engineering, or platform/infra — using this year's breadth as your map.
- **Year 3:** Start teaching — write, mentor, do internal tech talks. Explaining is the fastest way to find gaps.

- **Year 4-5:** Read source code of major OSS Java projects (Spring, Kafka, Netty) rather than blogs. Contribute real fixes.
- Re-run this 365 list once a year — you'll notice how differently you read the same topic titles.